

Graph Preconditioning for High-Dimensional Fixed Effects Regression

Alexander Fischer¹ and Kristof Schröder²

Draft: August 8, 2026

Abstract. The Method of Alternating Projections (MAP) is the standard algorithm for estimating high-dimensional fixed-effect regressions. It is fast when the fixed-effect dimensions are well connected, but converges slowly on sparse or nearly nested fixed-effect graphs, such as matched employer-employee panels where worker-firm mobility links separate worker and firm effects. MAP convergence depends on the worker-firm mobility pattern, but the algorithm uses that information only indirectly by iterating over one fixed effect at a time. The matrix of worker-firm match counts records the pattern directly. Together with the worker and firm count diagonals, it forms a graph Laplacian after a sign flip. A graph Laplacian is the standard matrix representation of a weighted graph. The graph structure lets us approximate the Laplacian’s inverse using sparse matrices. We propose a graph-preconditioned iterative solver with a reusable preconditioner built from factor-pair subproblems, such as worker-firm and worker-year pairs. These subproblems use the graph directly. Benchmarks show that all methods are fast on well-connected designs. When connectivity weakens, MAP and diagonally preconditioned LSMR slow down. Factor-pair preconditioning remains fast, and its runtime falls because the graph preconditioner is cheaper to construct on sparser graphs.

Keywords: high-dimensional fixed effects; alternating projections; preconditioning; matched employer-employee data; computational econometrics

JEL codes: C55; C63; C81; C87; J31

1. Introduction

Fixed effects appear in roughly half of the research published in leading economics and finance journals, according to Goldsmith-Pinkham (2026). Across applied economics, labor economists use worker and firm fixed effects to separate worker heterogeneity from firm wage premia; health economists study physician practice styles with individual, physician, and region fixed effects in mover designs; and education researchers use school, student, teacher, and student-teacher effects.

The Frisch-Waugh-Lovell (FWL) theorem reduces estimation to two operations: remove the variation explained by the fixed effects from the outcome and regressors, then run a low-dimensional regression on the residualized variables (Frisch and Waugh 1933; Lovell 1963). With high-dimensional fixed effects, software usually performs the first operation with the Method of Alternating Projections (MAP), also known as iterative demeaning or the “Zig-Zag” algorithm (Guimaraes and Portugal 2010; Gaure 2013).

¹trivago

²appliedAI Institute for Europe gGmbH

Packages including Stata’s `reghdfe` (Correia 2014; 2017), R’s `fixest` (Bergé et al. 2026), and Python’s `PyFixest` (PyFixest Developers 2026) use MAP or accelerated variants of it.

MAP handles the fixed-effect dimensions one at a time and never uses the observed worker-firm links directly. In the worker-firm wage model of Abowd et al. (1999), extended here with year effects, MAP subtracts worker means, then firm means, then year means. Each update uses only the residual left by the previous update, and MAP repeats the sequence until changes in the residuals fall below a chosen tolerance.

Yet those links matter for identification, precision, and computation (Correia 2017; Jochmans and Weidner 2019) because they form a graph in which movers create paths between firms, while stayers add observations without connecting firms. When few workers move between groups of firms, some combinations of worker and firm effects become difficult to separate; MAP can then require many repetitions before estimates in one group reflect changes in another. The same mobility links that identify worker and firm effects thus govern how quickly MAP converges.

Following Jochmans and Weidner (2019), we measure the connectivity of a connected factor-pair graph by λ_2 , the second-smallest eigenvalue of its normalized Laplacian. We weight the graph by the number of observed co-occurrences, with smaller values indicating weaker connectivity. The measure always describes the connectivity of two fixed-effect dimensions, and unless stated otherwise, λ_2 refers to the worker-firm graph.

The same graph appears in the weighted cross-product matrix of the fixed-effect indicators, which we call the Gramian (Correia 2017). Its diagonal blocks count observations for each worker, firm, or other fixed-effect level, whereas its off-diagonal blocks count which levels are observed together, such as the number of observations for each worker-firm pair. MAP uses the diagonal blocks one at a time and does not use these pairwise counts directly.

We therefore build a factor-pair graph preconditioner for designs with a small pairwise λ_2 , where MAP tends to converge slowly. A preconditioner transforms a linear system so that an iterative solver reaches the same solution in fewer steps. Rather than handling the fixed-effect dimensions one at a time, our preconditioner builds a local problem for every pair, such as worker-firm and worker-year, and uses the observed links directly. After a sign change, each pair block is a graph Laplacian, the standard matrix representation of a weighted graph. Sparse matrix methods can approximate its inverse at low cost (Spielman and Teng 2014; Gao et al. 2025), and we use a weighted sum of these approximate pair solutions as the preconditioner for LSMR.³

Constructing the factor-pair preconditioner takes time. On a well-connected graph, the reduction in iterations may not repay this setup cost because MAP and diagonal preconditioning are already fast. As λ_2 falls, however, the reduction in iterations can outweigh the construction cost. Figure 1 compares total regression times as worker-firm connectivity varies, using the top row for worker mobility and the bottom row for sorting among movers. Within each row, the left panel compares package defaults,

³`FixedEffectModels.jl` (FixedEffectModels.jl Developers 2026) also uses LSMR (Fong and Saunders 2011) but applies diagonal preconditioning, which uses fixed-effect counts rather than links between fixed-effect dimensions. We instead precondition LSMR with the factor-pair graph.

whereas the right panel compares solvers within PyFixest. We choose each solver’s tolerance using the accuracy comparisons in Section 7 and apply those values throughout Figure 1.

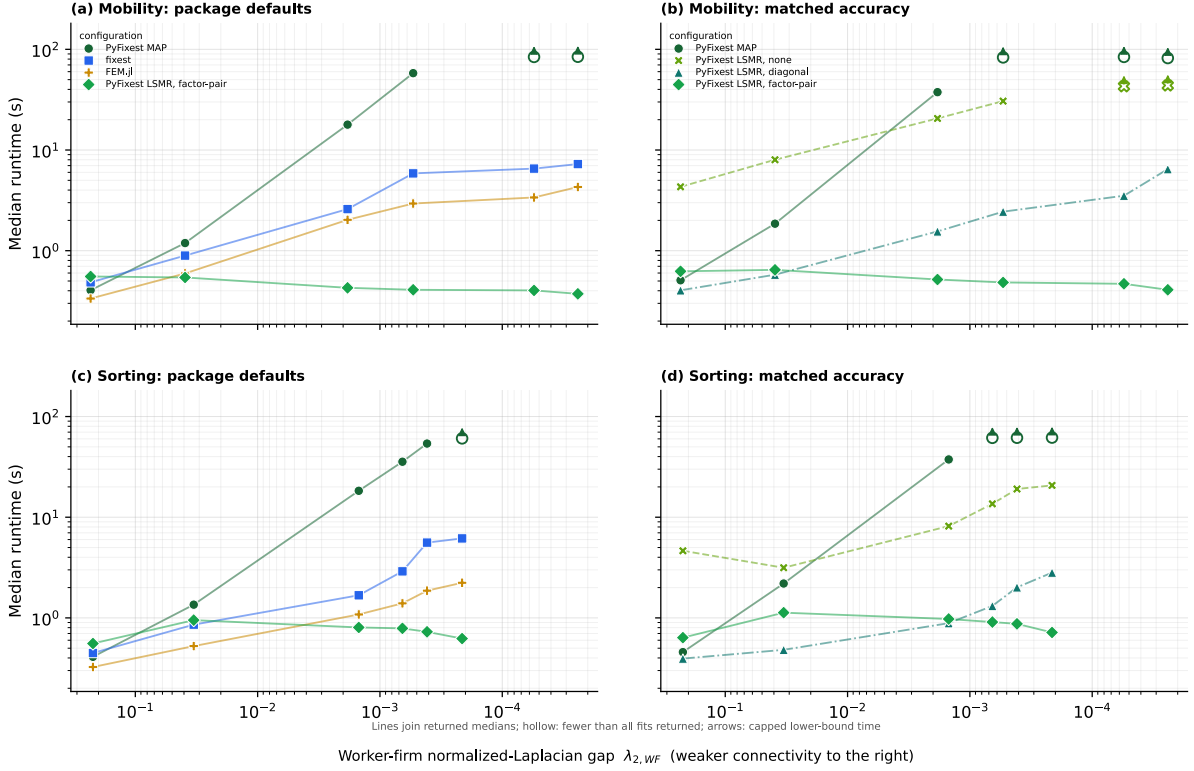


Figure 1: The panels plot median regression time in seconds against the worker-firm normalized-Laplacian gap $\lambda_{2,WF}$ on logarithmic axes. Here λ_2 is the second-smallest eigenvalue of the normalized Laplacian. We compute it after removing fixed-effect levels observed only once. For a disconnected graph, we compute λ_2 in each connected component and plot the smallest value. Smaller values mean weaker connectivity. All panels use the same reversed horizontal scale, so λ_2 decreases from left to right. Each simulated worker-firm-year panel has 1 million observations. The top panels vary worker mobility; the bottom panels hold move probability at one and vary sorting. The left panels compare packages at their default settings, including their own treatment of fixed-effect levels observed only once. The right panels compare four PyFixest solver configurations at tolerances calibrated in Section 7 to give comparable coefficient and residual errors. Filled markers indicate that all three planned fits produced estimates; hollow markers indicate that only one or two did. Lines join the medians in order of λ_2 and are not fitted trends. An arrow marks the median elapsed time for fits that reach the iteration limit. Because these fits did not finish, the marked time is a lower bound. Arrows are not joined to the lines. Fits that end in another error are omitted. At high connectivity, all implementations finish quickly. As λ_2 falls, however, MAP and LSMR without factor-pair preconditioning slow down, whereas factor-pair LSMR remains fast. In the lowest-mobility designs, its run time falls because the worker-firm subproblems are cheaper to construct.

Sections 2-5 set up fixed-effect absorption and connect MAP convergence to graph connectivity; Section 6 develops the factor-pair preconditioner, and Section 7 reports the benchmarks. Section 8 describes the software, Section 9 concludes, and the appendices give the algorithm, report additional benchmarks, and compare memory use.

2. Absorbing Fixed Effects⁴

We study the linear model

$$y = X\beta + D\alpha + \varepsilon, \quad (1)$$

⁴Researchers employ several names for this operation: “absorbing fixed effects”, “demeaning”, “residualizing”, or applying the “within transformation”. We use these terms interchangeably throughout.

where X contains the regressors of interest, D is the fixed-effect design matrix, and α collects the fixed-effect coefficients. Because D may have hundreds of thousands or millions of columns in high-dimensional applications, forming $[X \ D]$ or inverting its cross product can be computationally infeasible. The Frisch-Waugh-Lovell (FWL) theorem lets us compute $\hat{\beta}$ without either operation.

FWL reduces the computation of $\hat{\beta}$ to two steps. In the first, we regress y and each column of X separately on D and retain the residuals. Let M_D denote the linear operator that maps a variable to its residual, so that $\tilde{y} = M_D y$ and $\tilde{X} = M_D X$. In the second step, we regress the residualized outcome on the residualized covariates,

$$\tilde{y} = M_D y, \quad \tilde{X} = M_D X, \quad \hat{\beta} = (\tilde{X}' W \tilde{X})^{-1} \tilde{X}' W \tilde{y}, \quad (2)$$

where W is a diagonal matrix of weights. With one covariate, this amounts to three regressions: y on D , x on D , and $\tilde{y}$ on $\tilde{x}$. FWL implies that the slope in the last regression equals the coefficient on x in the full regression of y on x and D .

To residualize the outcome and each covariate in X , we project them onto the column space of the fixed-effect dummy matrix D by weighted least squares. For a variable μ to be residualized - either y or one column of X - we solve

$$\hat{\alpha}_\mu = \arg \min_{\alpha} \| D\alpha - \mu \|_W^2, \quad (3)$$

and the residual is $\tilde{\mu} = \mu - D\hat{\alpha}_\mu$. The first-order condition for Equation 3,

$$D'W(D\hat{\alpha}_\mu - \mu) = 0, \quad \text{equivalently} \quad G\hat{\alpha}_\mu = D'W\mu, \quad G = D'WD, \quad (4)$$

determines $\hat{\alpha}_\mu$. Its coefficient matrix G is the same for each FWL residualization; only the variable being residualized changes, so the computational cost depends on the structure of the Gramian G , the weighted cross-product matrix of the fixed-effect dummies. In the AKM worker-firm model, this structure has a concrete interpretation: worker moves link firms through shared workers and determine the off-diagonal blocks of G .

3. A Running Example: The AKM Model

Throughout the paper, we use the AKM model of Abowd et al. (1999) as the worker-firm example, in which workers observed at more than one firm provide the comparisons needed to distinguish persistent worker heterogeneity from firm wage premia. We write the regression as

$$y_{it} = \alpha_i + \psi_{J(i,t)} + \varphi_t + x'_{it}\beta + \varepsilon_{it}, \quad (5)$$

where α_i is a worker fixed effect, $\psi_{J(i,t)}$ is the fixed effect for the firm employing worker i at time t , and φ_t is a time fixed effect.

The AKM specification can be represented as a bipartite graph, with workers and firms as nodes and employment spells as edges. Worker moves induce a firm-to-firm graph by linking firms that employ the same worker; stayers - workers who never change employer - add observations to existing worker-firm links but do not bridge firms. Year effects enter as a third, low-dimensional factor on the same records, and Figure 2 contrasts a well-connected mobility graph with one that fragments under strong sorting.

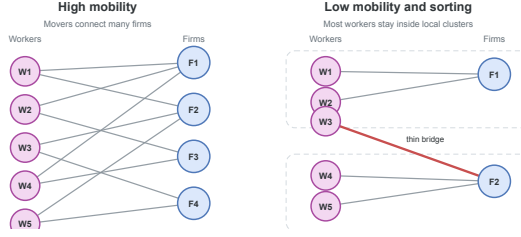


Figure 2: Worker-firm graphs under high and low mobility. High mobility creates many paths between firms. With low mobility and strong sorting, only a few worker moves connect otherwise separate groups of firms.

The same mobility links that affect computation provide the comparisons needed to identify worker and firm effects separately. A worker observed at only one firm provides no such comparison: a high wage could reflect an unusually productive worker, a high-wage firm, or both, so without moves the two components are not separately identified. Movers observed at firms with different wage profiles help attribute wage variation to worker effects or firm premia: high wages across several firms point toward a worker effect, whereas higher wages earned by many workers at the same firm point toward a firm premium.

As the number of such cross-firm comparisons grows, especially between otherwise weakly connected firms, worker and firm premia become easier to identify. In the graph, moves form the edges that connect firms; in the Gramian, those same links appear in the worker-firm cross-tabulation.

4. The Graph Structure of the Gramian

The block structure of the Gramian $G = D'WD$ represents the bipartite graph of worker and firm connections introduced in Section 3 (Correia 2017). Suppose that the columns of D are ordered as worker levels, firm levels, and year levels. Then

$$G = \begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix}. \quad (6)$$

The **diagonal blocks** G_{WW} , G_{FF} , and G_{YY} contain weighted counts for workers, firms, and years. An observation belongs to one level of each factor, so these blocks are diagonal; solving them requires only division by group counts.

The **off-diagonal blocks** are cross-tabulations: the worker-firm block C_{WF} records how often worker i is observed at firm j , and the worker-year and firm-year blocks have analogous interpretations.

The small worker-firm panel below illustrates how its Gramian is constructed. We set all regression weights to one, so $W = I$.

Obs.	Worker	Firm	Year	y
1	W_1	F_1	Y_1	3.2
2	W_1	F_2	Y_2	4.1
3	W_2	F_1	Y_1	2.8
4	W_2	F_1	Y_2	3.9
5	W_3	F_2	Y_1	5.0
6	W_3	F_2	Y_2	4.5

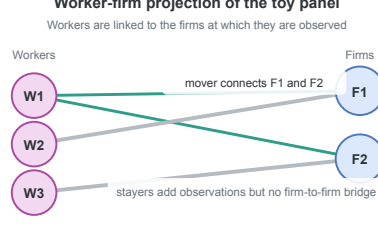


Figure 3: Worker-firm graph for the example panel. Worker W_1 works at both firms and connects them. Worker W_2 works only at F_1 , and worker W_3 works only at F_2 .

Figure 3 plots the worker-firm projection of this panel. Worker W_1 has employment spells at both F_1 and F_2 and therefore links the two firms, making W_1 a mover in AKM terms. By contrast, worker W_2 stays at F_1 for two periods and worker W_3 stays at F_2 for two periods; both are stayers.

The diagonal blocks are count matrices. In this example, each worker is observed twice, each firm three times, and each year three times, so

$$G_{WW} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{pmatrix}, \quad G_{FF} = \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}, \quad G_{YY} = \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}. \quad (7)$$

The off-diagonal blocks are cross-tabulations between factors. The worker-firm block is

$$C_{WF} = \begin{pmatrix} 1 & 1 \\ 2 & 0 \\ 0 & 2 \end{pmatrix}. \quad (8)$$

The first row shows that worker W_1 appears once at firm F_1 and once at firm F_2 . Workers W_2 and W_3 are stayers, appearing twice at F_1 and F_2 , respectively. The worker-year block is

$$C_{WY} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix}. \quad (9)$$

Each worker is observed once in each year. The firm-year block is

$$C_{FY} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}. \quad (10)$$

Firm F_1 appears twice in year Y_1 and once in year Y_2 ; firm F_2 has the opposite pattern. Together, the diagonal count blocks and off-diagonal cross-tabulation blocks form the full Gramian.

With column order $(W_1, W_2, W_3, F_1, F_2, Y_1, Y_2)$, the full Gramian is

$$G = \begin{pmatrix} 2 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 2 & 0 & 2 & 0 & 1 & 1 \\ 0 & 0 & 2 & 0 & 2 & 1 & 1 \\ 1 & 2 & 0 & 3 & 0 & 2 & 1 \\ 1 & 0 & 2 & 0 & 3 & 1 & 2 \\ 1 & 1 & 1 & 2 & 1 & 3 & 0 \\ 1 & 1 & 1 & 1 & 2 & 0 & 3 \end{pmatrix}. \quad (11)$$

The worker-firm submatrix encodes the bipartite graph: its diagonal entries are worker and firm counts, and the entries of C_{WF} are edge multiplicities between workers and firms. Changing the sign of C_{WF} turns this submatrix into a graph Laplacian, whose off-diagonal entries are non-positive and whose rows

sum to zero because each diagonal count cancels the off-diagonal spell counts in the same row. In a worker row, the diagonal entry gives the number of employment spells, while the off-diagonal entries show how many of those spells occur at each firm; firm rows have the analogous interpretation, with spell counts summed over workers.

$$L_{WF} = \left(\begin{array}{ccc|cc} 2 & 0 & 0 & -1 & -1 \\ 0 & 2 & 0 & -2 & 0 \\ 0 & 0 & 2 & 0 & -2 \\ \hline -1 & -2 & 0 & 3 & 0 \\ -1 & 0 & -2 & 0 & 3 \end{array} \right). \quad (12)$$

For any two fixed effects, this construction gives a pairwise Laplacian, and these Laplacians form the basis of the preconditioner in Section 6. MAP does not use them directly, but instead updates the diagonal worker, firm, and year blocks in turn, using the residual from one update as input to the next.

5. Alternating Projections and Graph Connectivity

The workhorse algorithm for multi-way fixed effects is the Method of Alternating Projections (MAP), also referred to as iterative demeaning or the “zig-zag” algorithm (Guimaraes and Portugal 2010; Gaure 2013), and many packages use MAP or a variant with acceleration methods (Berge 2018; Correia 2017). `fixest`, for example, uses Irons-Tuck extrapolation (Irons and Tuck 1969; Bergé et al. 2026). As Sections 3 and 4 show, MAP’s convergence rate depends on the fixed-effect graph and its algebra.

MAP solves the FWL residualization problem by iterating over one fixed effect at a time. In the worker-firm-year model, it subtracts worker means from the current residual, firm means from the updated residual, and then year means before repeating the sequence until convergence.

We write the FWL normal equations (see Equation 4) in block form with $D = [D_W \quad D_F \quad D_Y]$ as

$$\begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix} \begin{pmatrix} \alpha_W \\ \alpha_F \\ \alpha_Y \end{pmatrix} = \begin{pmatrix} D_{W'} W \mu \\ D_{F'} W \mu \\ D_{Y'} W \mu \end{pmatrix}. \quad (13)$$

Equivalently,

$$G_{WW}\alpha_W + C_{WF}\alpha_F + C_{WY}\alpha_Y = D_{W'} W \mu, \quad (14)$$

$$C_{(WF)'}\alpha_W + G_{FF}\alpha_F + C_{FY}\alpha_Y = D_{F'} W \mu, \quad (15)$$

$$C_{(WY)'}\alpha_W + C_{(FY)'}\alpha_F + G_{YY}\alpha_Y = D_{Y'} W \mu. \quad (16)$$

Rearranging each equation expresses one block of effects conditional on the others. Substituting $C_{WF} = D_{W'} W D_F$ and $C_{WY} = D_{W'} W D_Y$ into the first equation and factoring out $D_{W'} W$ gives

$$G_{WW}\alpha_W = D_{W'} W (\mu - D_F \alpha_F - D_Y \alpha_Y). \quad (17)$$

Because G_{WW} is diagonal, with entries equal to workers’ total observation weights, we obtain α_W by dividing each worker’s weighted partial residual by that worker’s total observation weight. For firms, the same rearrangement gives

$$G_{FF}\alpha_F = D_{F'} W (\mu - D_W \alpha_W - D_Y \alpha_Y), \quad (18)$$

and the year equation is analogous. Because G_{WW} , G_{FF} , and G_{YY} are all diagonal, we solve each equation by computing a weighted group mean in a single pass over observations.

MAP repeats the diagonal block updates while holding the other effects fixed. It updates the worker effects from the current partial residual before doing the same for firms and years; one complete pass cycles through the fixed-effect dimensions and subtracts the weighted group mean of the current partial residual for each factor.

The cross-tabulation blocks C_{WF} , C_{WY} , and C_{FY} enter the algorithm only indirectly: the worker update is computed from the partial residual $\mu - D_F\alpha_F - D_Y\alpha_Y$, while the firm update is computed from $\mu - D_W\alpha_W - D_Y\alpha_Y$. Rather than solving the worker-firm, worker-year, and firm-year subproblems jointly, MAP lets each block update change the residual used by the next update.

MAP converges quickly when the graph is well connected because high mobility creates overlapping employment histories that compare worker and firm effects. A high wage at one firm can then be related to wages earned by the same workers elsewhere, so a worker update changes partial residuals across many firms before the next firm update, and vice versa.

Sparse mobility, strong sorting, or near nesting has the opposite effect because movers form the edges of the worker-firm graph that identify worker and firm effects separately. MAP uses those links only through successive residual updates, so it may need many updates when they are few or concentrated in narrow regions of the graph, even though each complete pass remains cheap: the diagonal block updates reduce to one-pass group means.

We use the normalized-Laplacian gap $\lambda_{2,qr}$ as a diagnostic for MAP difficulty, with smaller values indicating weaker pairwise connectivity. For models with three or more fixed effects, the statistic describes one factor pair and does not bound convergence of the full model, whereas our preconditioner incorporates the pairwise links themselves.

6. The Factor-Pair Schwarz Preconditioner

6.1. Preconditioners

Unlike MAP, LSMR accepts a preconditioner that can use the pairwise links. MAP updates one fixed-effect dimension at a time, using its group labels and weights. The cross-tabulations affect an update only through the residual passed from the previous one. We therefore replace factor-by-factor MAP demeaning with LSMR (Fong and Saunders 2011), an iterative least-squares algorithm that improves an initial guess through repeated residual corrections. Because changing solvers alone offers little gain on weakly connected designs, a useful speed gain also requires a preconditioner.

LSMR converges quickly when the data contain comparable amounts of information about different combinations of fixed effects. Weak links, sparse mobility, or near nesting leave some combinations much less well determined than others, which makes the system poorly conditioned. A preconditioner transforms the normal equations to reduce these differences without changing the least-squares solution. The ideal preconditioner is G^{-1} .⁵ If $M^{-1} = G^{-1}$, then the matrix seen by LSMR is

⁵As in any model with several fixed effects, the level effects are pinned down only up to a normalization: we can add a constant to every worker effect and subtract it from every firm effect without changing the fitted values $D\alpha$, and likewise

$$M^{-1}G = G^{-1}G = I. \quad (19)$$

With this ideal preconditioner, LSMR would recover the solution after one correction because the inverse solves the system directly. Forming G^{-1} , however, would require solving the fixed-effect normal equations themselves, so the ideal inverse is a benchmark rather than an implementable preconditioner. A feasible preconditioner must instead approximate enough of G^{-1} to make poorly determined combinations of fixed effects easier for LSMR to resolve, and the iterations it saves must outweigh the time spent constructing and repeatedly applying it.⁶

6.2. From the Block Inverse to the Diagonal Preconditioner

The block structure of G^{-1} shows which parts of the ideal inverse a feasible preconditioner should retain.

For the worker-firm-year AKM model, the Gramian has the block form

$$G = \begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix}, \quad (20)$$

with diagonal weighted-count blocks G_{WW}, G_{FF}, G_{YY} and off-diagonal cross-tabulations C_{WF}, C_{WY}, C_{FY} . The standard formula for inverting a block matrix shows how each off-diagonal block enters G^{-1} . For the two-factor block

$$G_2 = \begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}, \quad (21)$$

it gives the explicit formula

$$G_2^{-1} = \begin{pmatrix} G_{WW}^{-1} + G_{WW}^{-1}C_{WF}S^{-1}C_{(WF)'}G_{WW}^{-1} & -G_{WW}^{-1}C_{WF}S^{-1} \\ -S^{-1}C_{(WF)'}G_{WW}^{-1} & S^{-1} \end{pmatrix}. \quad (22)$$

$$S = G_{FF} - C_{(WF)'}G_{WW}^{-1}C_{WF}. \quad (23)$$

The formula shows that C_{WF} enters through the Schur complement and through matrix products with S^{-1} ; the cross-tabulation itself is never inverted. Because G_{WW} is diagonal, applying G_{WW}^{-1} amounts to dividing by weighted worker counts. The expensive step is solving the firm-side system S . An exact factorization can create many additional nonzero entries, so its time and memory requirements depend on how many entries are added and on the number and size of the connected components. Weak mobility affects iterative methods differently: without a graph-aware preconditioner, they can be slow, but the sparser graph can make factorization cheaper because it has fewer links. Most of the work in the two-factor problem therefore lies in the Schur-complement system. The same calculation extends to three factors, where each block of G^{-1} can involve all three cross-tabulations C_{WF}, C_{WY}, C_{FY} .

The coarsest approximation to G^{-1} keeps only the diagonal count inverses and drops the Schur-complement corrections,

for years. The dummy-coded $G = D'WD$ and the smaller blocks inverted below are therefore not invertible until this ambiguity is removed - one normalization for the worker-firm pair, and a second once year effects are added, as in the Section 4 example. We adopt the standard normalization within each connected set and read every inverse below as that of the resulting system. The estimated effects depend on the normalization; the residualized outcome and regressors, which are all the regression uses, do not.

⁶LSMR never constructs $M^{-1}G$ or G explicitly. It only multiplies vectors by D and D' and applies M^{-1} .

$$M_{\text{diag}}^{-1} = \text{diag}(G_{WW}^{-1}, G_{FF}^{-1}, G_{YY}^{-1}). \quad (24)$$

Applying this preconditioner requires only one division by weighted level counts and gives the diagonal preconditioner used with LSMR in `FixedEffectModels.jl` (Fong and Saunders 2011; `FixedEffectModels.jl` Developers 2026). Although diagonal scaling records how many observations a level carries, it does not record how that level connects to the rest of the labor market. Level counts can still be useful when employment is concentrated in a few large firms, such as Novo Nordisk in Denmark or Samsung in South Korea, because the size differences alone remove an important source of scale variation. Diagonal scaling cannot, however, distinguish firms that are broadly linked to other employers from those whose workers remain concentrated in a narrow corner of the mobility graph.

6.3. The Factor-Pair Schwarz Approximation

Additive Schwarz preconditioning approximates a large problem by solving smaller, overlapping problems and adding their corrections (Xu 1992; Toselli and Widlund 2005). Here, each smaller problem retains the pairwise connectivity that diagonal scaling drops. In the AKM case, the three problems cover workers and firms, workers and years, and firms and years, with each one including the cross-tabulation between its two factors. Adding their corrections in the full vector of fixed-effect coefficients accounts for the main pairwise links in the Gramian, while the outer LSMR iteration handles the remaining three-way coupling. Consider the worker-firm pair. Its local problem is the two-factor block from the Schur calculation,

$$\begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}. \quad (25)$$

Figure 4 compares this worker-firm pair block with the single diagonal block solved by a factor-level MAP update.

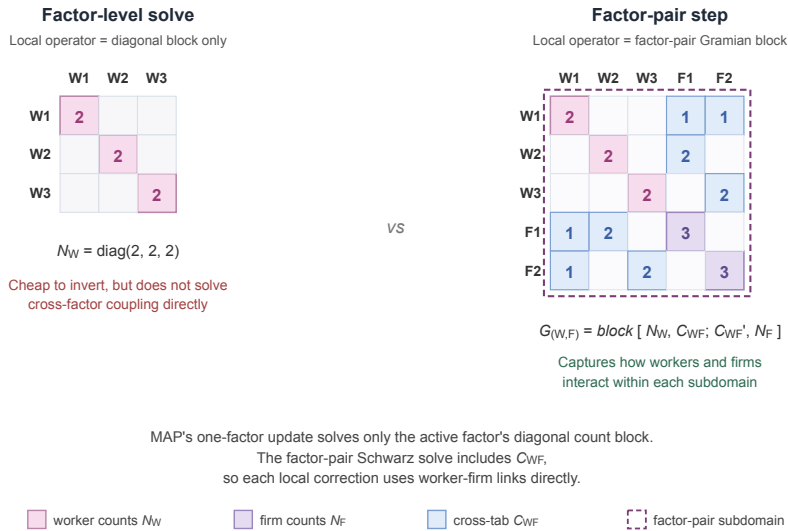


Figure 4: Matrices used by MAP and the factor-pair preconditioner in the example from Section 4. A MAP worker update (left) uses only the diagonal worker-count matrix $G_{WW} = \text{diag}(2, 2, 2)$. The factor-pair update (right) adds the firm-count matrix $G_{FF} = \text{diag}(3, 3)$ and the matrix of worker-firm observation counts C_{WF} . The dashed outline marks the worker and firm coefficients that are updated together. Its inverse carries C_{WF} through the Schur complement, so the local correction retains the worker-firm mobility geometry that diagonal scaling discards. To use this correction in the three-factor problem, let

R_{WF} select the worker and firm entries from the full coefficient vector $\alpha = [\alpha_W; \alpha_F; \alpha_Y]$; its transpose $R_{(WF)'}^T$ places the resulting correction back into the full vector. The diagonal matrix $\tilde{D}_{WF}$ weights levels that appear in more than one pair problem with partition-of-unity weights, so their corrections are not counted twice. The exact worker-firm contribution is

$$P_{WF}^{-1} = R_{(WF)'} \tilde{D}_{WF} \begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'}^T & G_{FF} \end{pmatrix}^{-1} \tilde{D}_{WF} R_{WF}. \quad (26)$$

The worker-year and firm-year contributions P_{WY}^{-1} and P_{FY}^{-1} are built the same way from G_{WW}, C_{WY}, G_{YY} and G_{FF}, C_{FY}, G_{YY} . With three factors each level appears in exactly two pair problems. Because the weights act on both sides of each pair contribution, we set them so that the squared weights on a shared level sum to one; a level in two pairs then carries $\frac{1}{\sqrt{2}}$. The exact factor-pair Schwarz preconditioner is the sum of these three contributions,

$$P^{-1} = P_{WF}^{-1} + P_{WY}^{-1} + P_{FY}^{-1}. \quad (27)$$

The three terms include the worker-firm, worker-year, and firm-year cross-tabulations separately. The simultaneous worker-firm-year coupling remains for the outer LSMR iteration, along with the error introduced by splitting shared levels across pairs.

6.4. Approximating Pair Systems via Graph Laplacians

For P^{-1} to serve as M^{-1} inside LSMR, we invert a pair block directly only when the two factors have few levels. In the Section 4 worker-firm example, the block has three worker levels and two firm levels; after the normalization in Section 6.1 removes the one free constant, the local calculation is a 4×4 inversion. Modern worker-firm register data can have hundreds of thousands of levels on each side, so direct inversion would recreate the large linear-algebra problem that the preconditioner is meant to avoid. We instead use the pair block's graph-Laplacian structure.

For a worker-firm pair, the local pair step solves the pair-Gramian system

$$\begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'}^T & G_{FF} \end{pmatrix} x = u, \quad (28)$$

where u is obtained by selecting and weighting the worker and firm entries of the vector that LSMR passes to the preconditioner. This block is not a graph Laplacian because its off-diagonal entries C_{WF} are non-negative. Let $T_{WF} = \text{diag}(I_W, -I_F)$ flip the sign of the firm entries, so that $T_{WF}^2 = I$. Multiplying the pair block by T_{WF} on both sides gives

$$L_{WF} = T_{WF} \begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'}^T & G_{FF} \end{pmatrix} T_{WF} = \begin{pmatrix} G_{WW} & -C_{WF} \\ -C_{(WF)'}^T & G_{FF} \end{pmatrix}, \quad (29)$$

a weighted bipartite graph Laplacian: symmetric, with non-positive off-diagonals and zero row sums. Because $T_{WF}^2 = I$, the pair-Gramian solution follows from the Laplacian solution by the same flip on each side,

$$\begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'}^T & G_{FF} \end{pmatrix}^{-1} = T_{WF} L_{WF}^{-1} T_{WF}, \quad (30)$$

where both inverses are read as in Section 6.1. We fix the free constant on each connected component by returning the zero-mean solution; the residualized variables do not depend on this choice. Solving the Laplacian once therefore yields the pair-Gramian solution: we flip the right-hand side, apply L_{WF}^{-1} , and flip the result back.

For preconditioning, the local system need not be solved exactly because the outer LSMR iteration refines the remaining error. We solve the Laplacian approximately using sparse Cholesky factorizations (Spielman and Teng 2014; Gao et al. 2025). An exact factorization would gradually make the sparse pair block denser: eliminating a worker creates new nonzero entries linking every pair of firms that worker visited, and repeated elimination causes these entries to accumulate. This fill-in raises the cost of storing and updating the factorization; in the worst case, a k -level system requires roughly k^3 operations and memory proportional to k^2 , as in a dense factorization. The randomized approximation controls this growth, with a cost roughly proportional to the number of observed worker-firm links apart from factors that grow only logarithmically with problem size. After transforming the approximate Laplacian solution back to worker-firm coordinates, we write A_{WF} for the resulting approximate pair-Gramian inverse.

Applying this construction to the worker-year and firm-year pairs yields A_{WY} and A_{FY} . Replacing the exact pair inverses in the Schwarz sum with these approximations gives the implemented preconditioner,

$$M^{-1} = \sum_{(q,r)} R_{(qr)'} \tilde{D}_{qr} A_{qr} \tilde{D}_{qr} R_{qr}. \quad (31)$$

In the worker-firm-year case each factor receives a diagonal contribution from its two pair subdomains and each off-diagonal correction from the corresponding pair.

The exact P^{-1} replaces the full three-factor inverse with exact pair inverses, while M^{-1} also approximates each pair inverse through A_{qr} . Both approximations change the update at each LSMR iteration but leave the original fixed-effect least-squares problem unchanged. LSMR therefore returns the same fitted residuals as an unpreconditioned LSMR run, up to the requested tolerance.

6.5. Implementation Strategy

Figure 5 summarizes the construction. We first divide the fixed-effect graph into overlapping factor pairs and use a sign flip to turn each local Gramian block into a graph Laplacian. A sparse approximate Cholesky routine then approximates each pair inverse while limiting the additional nonzero entries created during factorization. Finally, we weight the overlapping corrections so that shared fixed-effect levels are not counted twice and add them to form the Schwarz preconditioner M^{-1} .

The outer LSMR iteration uses this preconditioner to choose better-scaled updates (Fong and Saunders 2011; Arridge et al. 2014; Yang et al. 2024); once constructed, it can be reused to residualize the outcome and every covariate. Appendix A gives the algorithmic details.

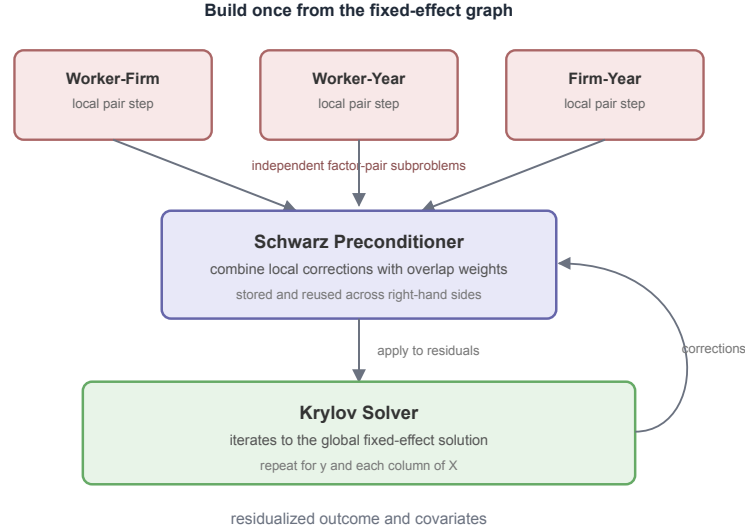


Figure 5: Construction of the factor-pair preconditioner. Each local problem combines two fixed-effect dimensions and retains their observed links. A sign change turns its matrix into a graph Laplacian, the standard matrix representation of a weighted graph. Sparse approximate Cholesky solves each local system while limiting the extra nonzero entries created during factorization, which saves memory and computation. The weighted sum of these local solutions is the preconditioner for LSMR.

7. Benchmarks

When connectivity is weak, MAP may require many factor updates to separate the fixed effects. Factor-pair preconditioning should help most on these graphs, although it may add unnecessary setup cost on well-connected ones; we test this prediction on controlled synthetic designs and public benchmark datasets.

The main runtime tables use package-level regression APIs, as does the public PyFixest benchmark suite, rather than timing the demeaning step alone. Each timing covers the full regression workflow: model setup, construction of the fixed-effect representation, residualization of the outcome and covariates, and estimation of the coefficient of interest. Appendix C reports the memory cost of storing factor-pair information.

For cross-package comparisons, every software implementation receives the same input data but uses its own default rules for singleton fixed-effect levels (levels that occur only once) and observations removed because of separation. We record the retained-observation count after successful fits and the convergence status and error after failed attempts; any shared control is stated with the relevant table. The package comparisons are therefore not constrained to a common estimation sample, whereas the later single-package experiments use a common prepared sample to isolate the preconditioner.

The runtime tables report λ_2 for the relevant factor pair after removing singleton fixed-effect levels. For a disconnected pair graph, we compute λ_2 within each connected component and report the smallest value, followed in parentheses by that component’s share of retained observations. Pairwise λ_2 remains a diagnostic rather than a convergence bound for models with three or more fixed effects.

The OLS package-runtime tables compare six configurations: PyFixest MAP, PyFixest LSMR with no, diagonal, or factor-pair preconditioning, R `fixest`, and `FixedEffectModels.jl`:

Configuration	Package	Algorithm
rust-map	PyFixest	Vanilla Rust MAP, without acceleration.
within-off	PyFixest	LSMR without preconditioning.
within-diagonal	PyFixest	LSMR with diagonal preconditioning.
within	PyFixest	LSMR with factor-pair Schwarz preconditioning.
fixest	R fixest	Accelerated MAP with Irons-Tuck and other optimizations (Bergé et al. 2026).
FEM.jl	FixedEffectModels.jl	Diagonally preconditioned LSMR (Fong and Saunders 2011; FixedEffectModels.jl Developers 2026).

Reported CPU times are medians over measured repetitions: the main OLS benchmarks use the adaptive R1/R2/R3 rule, while experiments with a fixed count use three repetitions. All runs use an Apple M4 Mac mini with 10 CPU cores and 16 GB of memory running macOS 15.3.1.

We do not include reghdfe (Correia 2014; 2017) directly in the benchmark tables because Stata is not open source and we lack a license. reghdfe is a mature accelerated-MAP implementation; algorithmically, it belongs to the same family as fixest’s accelerated MAP.

7.1. Runtime Benchmarks

Because the implementations use different stopping rules, their reported tolerance settings are not directly comparable. We first compare package defaults; later in this section, the achieved-precision experiment evaluates every method using the same measures of coefficient and residual error.

The main OLS benchmarks use synthetic AKM-style panels with one million observations, one covariate, ten periods, and worker, firm, and year fixed effects. Holding the rest of the data-generating process fixed, we vary the worker-firm graph in two ways: the mobility designs progressively reduce the number of workers who change firms, whereas the sorting designs hold the move probability at one but assign a larger share of moves within groups of firms. Appendix B reports the exact timings behind Figure 1.

7.1.1. Worker Mobility

Lower worker mobility leaves fewer workers connecting multiple firms. Because MAP updates one fixed-effect dimension at a time, a worker update affects firm coefficients only through the residual used in the next firm update, whereas the factor-pair preconditioner solves worker-firm blocks jointly. Its relative advantage should therefore grow as mobility and worker-firm λ_2 fall; across the designs, λ_2 declines from 0.232 to 2.41×10^{-5} . Under package defaults, all configurations in the first two designs complete within 3.60 seconds, but PyFixest MAP then slows sharply or reaches its cap, and LSMR without preconditioning reaches its cap. Factor-pair LSMR instead falls from 0.554 seconds in the first design to 0.373 seconds in the last (top row of Figure 1).

7.1.2. Sorting Among Movers

Every worker changes firms between adjacent periods in these designs, but the sorting parameter ρ controls how strongly workers are matched to similar firms; larger values concentrate moves within groups and weaken the links between them. As ρ rises from 0 to 150,000, worker-firm λ_2 falls from 0.222 to 2.13×10^{-4} . MAP rises from 0.411 seconds at $\rho = 0$ to 53.9 seconds at $\rho = 10,000$ and then reaches its default cap at $\rho = 150,000$; LSMR without preconditioning reaches its default cap in the last four designs. Diagonal LSMR rises from 0.362 to 1.58 seconds, whereas factor-pair LSMR remains below 1 second and

takes 0.623 seconds at $\rho = 150,000$; it is faster than diagonal LSMR in the three least-connected designs (bottom row of Figure 1).

7.1.3. When Preconditioner Setup Dominates

The public `fixest` benchmark suite complements the gradual connectivity changes in the AKM designs by contrasting a dense, well-connected design with a sparse, nearly nested design (Bergé et al. 2026). Both contain 10 million observations, one covariate, and worker, firm, and year fixed effects.

Simple and difficult `fixest` designs (10 million observations, three fixed effects).

Design	λ_2 (share)	PyFixest MAP	PyFixest LSMR none	PyFixest LSMR diagonal	PyFixest LSMR factor-pair	fixest	FEM.jl
simple (well-connected)	0.622 (1.00)	2.35s	2.72s	2.16s	11.2s	2.52s	2.08s
difficult (near-nested)	8.35×10^{-8} (1.00)	326.2s	10.2s	capped (0/3)	4.63s	62.2s	27.0s

Note: Times are in seconds and cover each OLS regression from model setup through coefficient estimation. Each package uses its default settings. A preliminary run sets the number of measured runs: 20 if it takes less than 1 second, 7 if it takes 1 to 10 seconds, and 3 if it takes longer. When only k of n planned runs return an estimate, $t(\frac{k}{n})$ reports their median time t . A dash means that no timing is available. capped (0/ n) means that no run finishes before the iteration limit; failed (0/ n) marks a failure other than reaching that limit. Both designs have 10 million observations, one covariate, and worker, firm, and year fixed effects. In the λ_2 (share) column, λ_2 is the second-smallest eigenvalue of the normalized Laplacian for the worker-firm graph. We compute it after removing fixed-effect levels observed only once; smaller values mean weaker connectivity. For a disconnected pair graph, we compute λ_2 in each connected component and report the smallest value. The number in parentheses is that component's share of retained observations.

On the dense design, all methods except factor-pair LSMR finish in 2.08 to 2.72 seconds; factor-pair LSMR takes 11.2 seconds because its setup cost is not recovered in one fit. On the near-nested design, factor-pair LSMR takes 4.63 seconds, compared with 10.2 seconds without preconditioning, 27.0 seconds for FEM.jl, 62.2 seconds for `fixest`, and 326.2 seconds for PyFixest MAP, while PyFixest diagonal LSMR reaches its default cap.

7.1.4. Iterations After Setup

The iteration-count diagnostic separates construction from the subsequent LSMR work and uses the 100,000-observation versions of the same simple and difficult designs.

Iterations on the simple and difficult designs (100,000 observations).

Design	map (passes)	off	diagonal	additive
simple	14	37	16	14
difficult	8159	249	180	22

Note: The MAP column counts complete passes over all fixed-effect dimensions. The other columns count LSMR iterations: `off` uses no preconditioner, `diagonal` scales by fixed-effect group counts, and `additive` uses the factor-pair preconditioner. A MAP pass and an LSMR iteration perform different work, so their counts are not directly comparable. Each count is the median of three runs; each run removes the fixed effects from one outcome and one covariate.

Among the LSMR configurations, the additive preconditioner needs 14 iterations on the simple design and 22 on the difficult one, while the diagonal count rises from 16 to 180 and the unpreconditioned count from 37 to 249. Once the additive preconditioner has been built, the difficult design requires only eight more LSMR iterations than the simple design; despite these low iteration counts, setup makes within unattractive for the simple one-shot regression.

7.1.5. Public Real-Data Benchmarks

The controlled designs isolate mobility, sorting, and near nesting, but empirical fixed-effect graphs also contain large differences in group size, thin links between dense groups, and many disconnected components. The public benchmark collection assembled by Correia (2017) contains eight such datasets and lets us check whether the controlled comparisons carry over to these less regular graphs.

Correia real-data benchmarks.

Dataset	λ_2 (share)	PyFixest MAP	PyFixest LSMR factor-pair	fixest	FEM.jl
credit	0.227 (1.00)	0.193s	0.242s	0.154s	0.097s
soccer	0.667 (1.00)	0.026s	0.065s	0.014s	0.010s
enron	3.53×10^{-3} (0.99)	2.95s	0.402s	0.535s	0.317s
github	3.15×10^{-4} (0.32)	capped (0/3)	0.310s	1.43s	1.44s
patents	2.59×10^{-4} (0.95)	capped (0/3)	0.388s	1.05s	0.576s
workers	1.37×10^{-4} (0.63)	capped (0/3)	0.279s	2.00s	1.45s
schools	1.11×10^{-3} (1.00)	5.98s	0.266s	0.828s	0.487s
directors	2.56×10^{-4} (0.30)	capped (0/3)	0.327s	0.283s	0.462s

Note: Times are in seconds for OLS regressions using each package’s default settings, with three planned runs per cell. Each package applies its own rule for removing fixed-effect levels observed only once. If only k runs return an estimate, $t(\frac{k}{3})$ reports their median time t . capped (0/3) means that no run finishes before the 10,000-iteration limit. In the λ_2 (share) column, λ_2 is the second-smallest eigenvalue of the normalized Laplacian for the graph linking id1 and id2, computed after singleton removal. For a disconnected graph, we report the smallest component value; the number in parentheses is that component’s share of retained observations.

On *credit* and *soccer*, PyFixest MAP takes 0.193 and 0.026 seconds, compared with 0.242 and 0.065 seconds for factor-pair LSMR, so the preconditioner’s setup cost does not pay on these easy datasets. On the more difficult graphs, factor-pair LSMR takes 0.402 seconds on *enron* and 0.266 seconds on *schools*, compared with 2.95 and 5.98 seconds for PyFixest MAP. On *github*, *patents*, *workers*, and *directors*, PyFixest MAP reaches its iteration cap while factor-pair LSMR finishes in 0.279 to 0.388 seconds. Accelerated MAP can still be competitive, however; *fixest* takes 0.283 seconds on *directors*, slightly less than factor-pair LSMR’s 0.327 seconds.

Pairwise λ_2 does not by itself explain every real-data result. For *directors*, the component with the smallest reported value contains only 30 percent of the retained observations, and the remaining graph may be easier. We therefore treat λ_2 as a description of one factor pair rather than as a rule for choosing a solver.

7.2. Runtime and Achieved Precision

The methods apply their stopping tolerances to different quantities. PyFixest MAP defaults to 10^{-6} and stops when every observation-level residual changes by less than that amount after a full pass over the fixed-effect dimensions. R’s *fixest* also defaults to 10^{-6} , but applies the threshold to successive fixed-effect coefficient iterates. It accepts either a small absolute change or a small scaled relative change, and periodically makes the same comparison for the residual sum of squares. *FixedEffectModels.jl* passes its default tolerance of 10^{-6} as both LSMR stopping tolerances. *within* defaults to 10^{-8} and stops when either the estimated least-squares residual is at most 10^{-8} times the norm of the variable being residualized (the right-hand side), or a scaled measure of the remaining first-order-condition error falls below 10^{-8} .

Writing $A = W^{\frac{1}{2}}D$ and $r = W^{\frac{1}{2}}(\mu - D\alpha)$, this second quantity is $\frac{\|A^T r\|_2}{\|A\|_F \|r\|_2}$, based on LSMR's running norm estimates (Fong and Saunders 2011).

Figure 6 compares the methods on a common accuracy scale for mobility designs 1, 3, and 5, using the same sample for every method. Before timing, we repeatedly remove fixed-effect levels that occur only once, perform this preparation once for each one-million-observation design, and pass the resulting rows to every method. At each value of a package's own tolerance, we impose a common 10,000-iteration cap and time three fits; the tight reference uses factor-pair LSMR at tolerance 10^{-14} . The package defaults for the iteration cap are 10,000 for PyFixest MAP, fixest, and FixedEffectModels.jl, and 1,000 for within.

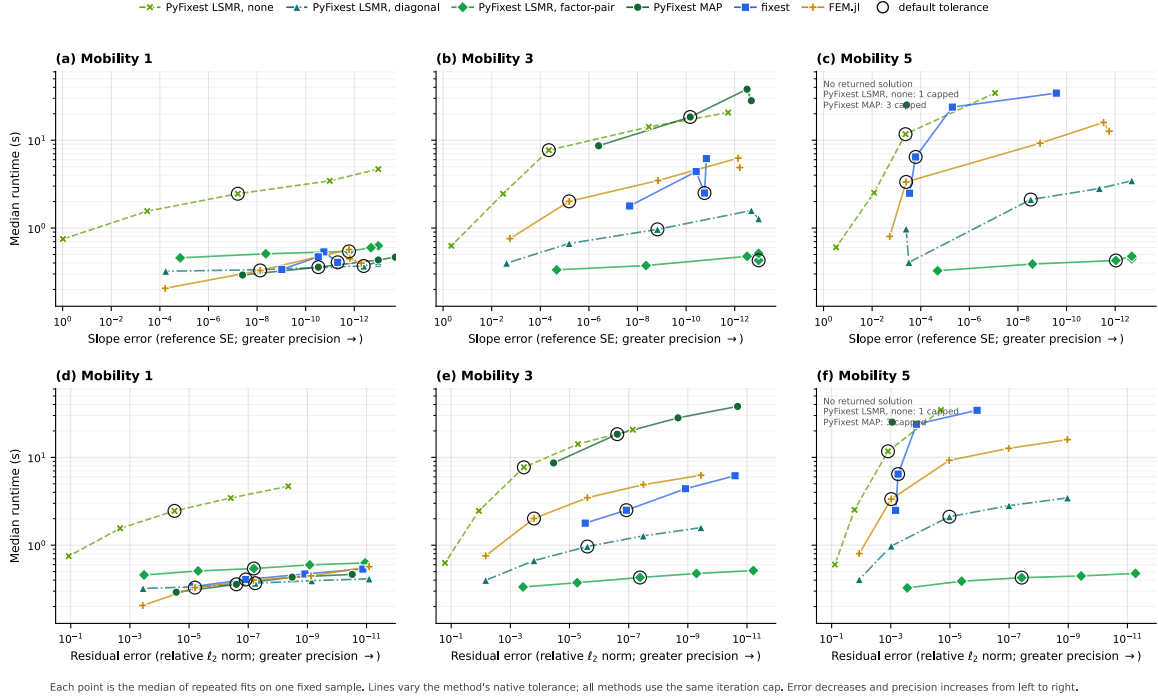


Figure 6: Elapsed time and achieved accuracy on three AKM mobility designs. Each marker reports the median elapsed time in seconds among the three planned fits that return an estimate on the same sample. Before timing, we remove fixed-effect levels observed only once and use the remaining observations for every method. Each line connects results for one method across requested tolerances. The top row measures coefficient error as $\frac{|\hat{\beta} - \hat{\beta}^*|}{SE(\hat{\beta}^*)}$. The bottom row measures residual error as $\frac{\|r - r^*\|_2}{\|r^*\|_2}$. The reference coefficient $\hat{\beta}^*$ and residual r^* come from factor-pair LSMR at tolerance 10^{-14} . Error decreases from left to right; on each line, a circle marks the result obtained with the package's default tolerance. Every fit is limited to 10,000 iterations. Settings for which none of the three fits returns an estimate are omitted and identified in the annotations.

7.3. Amortizing the Preconditioner

Comparing run time at achieved accuracy accounts for differences in stopping rules, but factor-pair LSMR still carries a fixed setup cost. The pair blocks and their approximate factorizations must be constructed before LSMR begins, although later regressions can reuse the resulting preconditioner when the observations, weights, and fixed-effect identifiers do not change.

7.3.1. Setup Cost Across Connectivity

We time construction and the subsequent LSMR work separately on the AKM mobility designs, using a two-factor specification with worker and firm effects and a three-factor specification that also absorbs year effects. We plan five runs per cell at one million observations, with the same outcome and covariate in both specifications.

Factor-pair setup and LSMR times as worker mobility varies.

Move probability δ	λ_2 (share)	2 FE setup	2 FE LSMR	3 FE setup	3 FE LSMR
1	0.232 (1.00)	0.109s	0.291s	0.113s	0.209s
0.5	0.0392 (1.00)	0.115s	0.255s	0.118s	0.234s
0.05	1.84×10^{-3} (0.87)	0.032s	0.135s	0.034s	0.233s
0.01	5.35×10^{-4} (0.61)	0.027s	0.124s	0.031s	0.216s
0.005	5.50×10^{-5} (0.52)	0.025s	0.093s	0.031s	0.193s
0.001	2.41×10^{-5} (0.29)	0.028s	0.045s	0.032s	0.149s

Note: Times are in seconds. For each design and specification, we plan five runs with 1 million observations and an LSMR tolerance of 10^{-12} . The table gives the median setup time and time spent in LSMR among the runs that finish; if only k finish, $t(\frac{k}{5})$ reports their median time t . The two-fixed-effect specification absorbs worker and firm effects; the three-fixed-effect specification also absorbs year effects. In the λ_2 (share) column, λ_2 is the second-smallest eigenvalue of the normalized Laplacian for the worker-firm graph. We compute it after removing fixed-effect levels observed only once; smaller values mean weaker connectivity. For a disconnected pair graph, we compute λ_2 in each connected component and report the smallest value. The number in parentheses is that component's share of retained observations.

Comparing the endpoints, setup is cheaper in the lowest-mobility design than in the highest, falling from 0.109 to 0.028 seconds with two fixed effects and from 0.113 to 0.032 seconds with three. Over the same comparison, time spent in LSMR falls from 0.291 to 0.045 seconds with two effects and from 0.209 to 0.149 seconds with three. The low-mobility worker-firm graph has fewer cross-firm links to store and factorize, so the additive runtime in Figure 1 can decline even as MAP and diagonal LSMR slow down.

7.3.2. Ten Regressions on the Same Fixed Effects

Many empirical projects estimate several specifications on the same sample and fixed effects, allowing the factor-pair objects to be constructed once and reused. We measure ten sequential regressions on both one-million-observation `fixest` designs, using the same outcome and ten covariates under all three policies; the two additive policies either construct a new preconditioner for every regression or keep one preconditioner for all ten.

Ten regressions with rebuilt and cached preconditioners.

Design	Policy	Setup	LSMR	Total	Speedup
simple	Diagonal	0.048s	0.553s	0.601s	1.0x
	Additive, rebuilt	2.80s	1.51s	4.29s	0.1x
	Additive, cached	0.281s	1.51s	1.79s	0.3x
difficult	Diagonal	0.049s	43.6s	43.6s	1.0x
	Additive, rebuilt	0.379s	1.20s	1.58s	27.5x
	Additive, cached	0.040s	1.26s	1.30s	33.6x

Note: For each policy, we add the setup and LSMR times across ten sequential regressions and take the median across three planned repetitions. If only k repetitions finish, $t(\frac{k}{3})$ reports their median time t , and the speedup is omitted. Times are in seconds. Each design has 1 million observations and worker, firm, and year fixed effects. Each regression removes the fixed effects from the common outcome and one of ten covariates at an LSMR tolerance of 10^{-12} . For complete cells, speedup divides the diagonal total time by the reported total time. `Additive, rebuilt` constructs a new factor-pair preconditioner for each regression; `Additive, cached` reuses one preconditioner for all ten.

Across ten regressions, the additive preconditioner remains slower on the simple design: diagonal preconditioning takes 0.601 seconds, compared with 4.29 seconds when the additive preconditioner is rebuilt and 1.79 seconds when it is cached. Caching avoids nine constructions, but the additive configuration remains more than twice as slow as diagonal preconditioning on this well-connected graph.

On the difficult design, diagonal preconditioning takes 43.6 seconds for the ten regressions, whereas rebuilding the additive preconditioner lowers the total to 1.58 seconds and caching lowers it further to

1.30 seconds. Caching reduces additive setup time from 0.379 to 0.040 seconds; time spent in LSMR is 1.20 seconds with rebuilding and 1.26 seconds with caching.

7.4. Poisson and Other GLMs

Faster fixed-effect residualization matters especially for generalized linear models fit by iteratively reweighted least squares (IRLS), because each IRLS step fits a weighted least-squares problem and demeans the response and covariates against the fixed effects (Stammann 2018). The mapping from observations to fixed-effect levels remains constant while the weights change, a pattern used by `ppmlhdfe` (Correia et al. 2020) and other IRLS-based estimators with high-dimensional fixed effects.

PyFixest currently reuses the preconditioner built during the first weighted demeaning call, avoiding a new factorization at every IRLS step even though later problems then use a preconditioner based on earlier weights. Reuse saves construction time but may require more LSMR iterations, whereas rebuilding may reduce the iteration count at the cost of a new construction at every step. The table reports the current PyFixest reuse behavior.

The PPML comparison uses the simple and difficult designs from the `fixest` benchmark suite (Bergé et al. 2026) at $n = 1\text{M}$ observations, with one covariate and worker, firm, and year fixed effects. The compared implementations are R `fixest`’s `fepois`, `GLFixedEffectModels.jl`, and two PyFixest `fepois` paths: the default unpreconditioned `rust-map` and `within` with PyFixest’s current reuse policy. Packages apply their default rules for separated observations, and each implementation receives the same 100-iteration outer IRLS limit.

Poisson benchmarks (1 million observations, one covariate).

Design	FE	PyFixest MAP	fixest	GLFEM.jl	PyFixest LSMR factor-pair
simple (well-connected)	3	7.95s	4.63s	5.71s	9.74s
difficult (near-nested)	3	capped (0/3)	439.3s	129.8s	5.52s

Note: Times are in seconds for Poisson fixed-effect regressions with 1 million observations, one covariate, and worker, firm, and year fixed effects. `fixest` is R `fixest::fepois`; `rust-map` and `within` use PyFixest `fepois`; and `GLFEM.jl` is `GLFixedEffectModels.jl`. The `within` configuration reuses the first factor-pair preconditioner as the weights change across iteratively reweighted least squares steps. Each package applies its default rule for removing separated observations, where a combination of regressors and fixed effects can push fitted means for some zero outcomes arbitrarily close to zero. If only k of the three planned fits return an estimate, $t(\frac{k}{3})$ gives their median time t . `capped (0/3)` means that no fit finishes before either the package’s inner demeaning limit or the common limit of 100 iteratively reweighted least squares steps; `failed (0/3)` marks another failure.

On the simple design, all four paths finish in under ten seconds: `fixest` takes 4.63 seconds, `GLFEM.jl` 5.71, `rust-map` 7.95, and `within` 9.74. On the difficult design, `rust-map` does not converge within the iteration cap; `GLFEM.jl` takes 129.8 seconds, `fixest` 439.3, and `within` 5.52. Sparse worker-firm coupling slows MAP, whereas factor-pair LSMR handles the worker-firm blocks jointly.

8. Software

The open-source `within` project provides the solver benchmarked in Section 7 (within Developers 2026), with a computational core written in Rust and APIs for Python and R.

Interface	Package	Registry	Install
Rust	within	crates.io	cargo add within
Python	within-py	PyPI	pip install within-py
R	withinr	py-econometrics R-universe	install.packages("withinr", repos = "https://py-econometrics.r-universe. dev")

The Rust crate exposes the lower-level solver and its configuration types, while the Python and R packages provide `solve` and `solve_batch` APIs for residualizing one or several variables. PyFixest offers the same algorithm as a demeaning option (PyFixest Developers 2026).

The Python example below demeans an outcome and two covariates against worker and firm fixed effects, then runs the FWL regression on the demeaned variables:

```
import numpy as np
from within import solve_batch
# Worker and firm identifiers as a column-major uint32 array
n = 100_000
categories = np.asfortranarray(np.column_stack([
    np.random.randint(0, 5_000, n).astype(np.uint32),
    np.random.randint(0, 500, n).astype(np.uint32),
]))
# Outcome and covariates
beta = np.array([1.0, -2.0])
X = np.random.randn(n, 2)
y = X @ beta + np.random.randn(n)
# Residualize y and X jointly; the preconditioner is reused across columns
res = solve_batch(categories, np.column_stack([y, X]))
y_tilde, X_tilde = res.demeaned[:, 0], res.demeaned[:, 1:]
# FWL on the demeaned variables
beta_hat = np.linalg.lstsq(X_tilde, y_tilde, rcond=None)[0]
```

`solve_batch` follows the FWL workflow from Section 2 by residualizing the outcome and covariates together and reusing one factor-pair preconditioner across columns, so users do not need to construct the Gramian or its pairwise blocks themselves.

9. Conclusion

The connections among the fixed effects affect both identification and the time needed to estimate the model. In our benchmarks, MAP is difficult to outperform on dense, well-connected graphs: each pass is cheap, while constructing the factor pairs adds overhead. With low mobility, strong sorting, or nearly nested effects, however, MAP separates the fixed effects only gradually, and the factor-pair preconditioner is often much faster. The Correia real-data benchmarks follow the same broad pattern: PyFixest MAP is faster on credit and soccer, but reaches its iteration cap on four of the six datasets with the smallest reported λ_2 values, all of which factor-pair LSMR finishes in less than 0.4 seconds.

Constructing the factor pairs becomes cheaper in the low-mobility AKM designs, and regressions that share the same sample, fixed effects, and weights incur this cost only once. Caching lowers additive runtime in both designs, although it beats diagonal preconditioning only on the difficult design: there, setup falls from 0.379 to 0.040 seconds and total time from 1.58 to 1.30 seconds.

PPML repeatedly calls the demeaning routine because each IRLS step solves a new weighted problem, and PyFixest reuses the first preconditioner as the weights change in our benchmark. More generally, factor-pair preconditioning is most useful when pairwise λ_2 is small, a fit is unexpectedly slow, or an estimator repeatedly removes high-dimensional fixed effects.

Appendix A: Factor-Pair Schwarz Algorithm

Algorithm 1 implements the construction shown in Figure 5.

Algorithm 1. Factor-Pair Schwarz Preconditioner

Inputs

- Observation-level factor codes for Q fixed-effect dimensions.
- Diagonal weights W and local solver settings.
- Vector p , with one entry for each fixed-effect coefficient, passed to the preconditioner by the current LSMR iteration.

Preconditioner setup

- Enumerate all unordered factor pairs (q, r) with $q < r$.
- For each pair, build weighted count blocks G_{qq} , G_{rr} and the weighted cross-tabulation C_{qr} .
- Split the induced bipartite graph into connected components and create one local pair problem (a Schwarz subdomain) s per component.
- If fixed-effect level j appears in c_j subdomains, store the partition weight $\omega_j = \frac{1}{\sqrt{c_j}}$.
- For each subdomain, sign-flip one side to obtain a local Laplacian, remove the constant part that cannot be identified within the component from the local right-hand side, and prepare the local system on the zero-mean subspace.
- Use a Schur-complement local solver: small reduced systems are solved directly, while larger reduced symmetric diagonally dominant (SDD) or Laplacian systems are solved with randomized approximate Cholesky.

Application during LSMR

- Initialize $z = 0$.
- For each subdomain s , form $h_s = \tilde{D}_s R_s p$.
- Compute the approximate local correction $u_s \approx A_s h_s$ on the zero-mean subspace.
- Accumulate $z \leftarrow z + R_{s'} \tilde{D}_s u_s$.
- Return $z = M^{-1}p$.

Appendix B: Detailed AKM Timings

Appendix B reports the exact timings behind Figure 1. Each regression has 1 million observations, one covariate, and worker, firm, and year fixed effects. The columns compare PyFixest MAP; PyFixest LSMR with no, diagonal, or factor-pair preconditioning; R fixest; and FixedEffectModels.jl. Every method uses its package defaults, and the reported time is the median of three planned runs.

We compute worker-firm λ_2 after removing fixed-effect levels observed only once. For a disconnected graph, the table reports the smallest value across connected components and gives that component's share of retained observations in parentheses. If only k runs return an estimate, $t(\frac{k}{3})$ reports their median time t . capped (0/3) means that no run finishes before the 10,000-iteration limit; failed (0/3) marks a failure other than reaching that limit. The script `scripts/paper_results.py` generates both tables from recorded benchmark output.

Worker Mobility

Mobility benchmark.

Move probability δ	λ_2 (share)	PyFixest MAP	PyFixest LSMR none	PyFixest LSMR diagonal	PyFixest LSMR factor-pair	fixest	FEM.jl
1	0.232 (1.00)	0.405s	2.48s	0.371s	0.554s	0.485s	0.335s
0.5	0.0392 (1.00)	1.19s	3.65s	0.466s	0.542s	0.894s	0.594s
0.05	1.84×10^{-3} (0.87)	17.9s	capped (0/3)	0.957s	0.428s	2.59s	2.03s
0.01	5.35×10^{-4} (0.61)	58.0s	capped (0/3)	1.34s	0.409s	5.86s	2.94s
0.005	5.50×10^{-5} (0.52)	capped (0/3)	capped (0/3)	2.18s	0.403s	6.54s	3.38s
0.001	2.41×10^{-5} (0.29)	capped (0/3)	capped (0/3)	3.16s	0.373s	7.25s	4.29s

Note: Times are in seconds and cover each regression from model setup through coefficient estimation. Move probability is the probability that a worker changes firms between adjacent periods.

Sorting Among Movers

Sorting benchmark.

Sorting strength ρ	λ_2 (share)	PyFixest MAP	PyFixest LSMR none	PyFixest LSMR diagonal	PyFixest LSMR factor-pair	fixest	FEM.jl
0	0.222 (1.00)	0.411s	2.60s	0.362s	0.557s	0.448s	0.325s
20	0.0333 (1.00)	1.35s	2.04s	0.427s	0.950s	0.857s	0.525s
500	1.49×10^{-3} (1.00)	18.3s	capped (0/3)	0.814s	0.804s	1.68s	1.08s
2000	6.55×10^{-4} (1.00)	35.6s	capped (0/3)	1.05s	0.787s	2.90s	1.39s
10000	4.12×10^{-4} (1.00)	53.9s	capped (0/3)	1.35s	0.727s	5.59s	1.86s
150000	2.13×10^{-4} (1.00)	capped (0/3)	capped (0/3)	1.58s	0.623s	6.16s	2.23s

Note: Times are in seconds and cover each regression from model setup through coefficient estimation. Every worker changes firms between adjacent periods; the parameter ρ controls how strongly workers are matched to similar firms.

Appendix C: Memory Use

MAP uses little memory because a complete pass needs only the current residuals and per-level group sums, whereas a factor-pair preconditioner must retain the pair structure between iterations. We therefore measure how much additional memory factor-pair LSMR requires relative to MAP.

We record peak resident set size (peak RSS), the largest amount of physical memory used by the process during a run, on the simple and difficult fixed-effect designs. The main storage terms are the data matrix, the fixed-effect encodings, and the reusable factor-pair objects. We do not compare `fixest` and `FixedEffectModels.jl` here because peak RSS across languages also reflects R and Julia runtime overhead, data-loading choices, garbage collection, and package internals. Instead, we compare the preconditioned Rust implementation with Rust MAP inside the same Python package, holding the surrounding regression code fixed so that only the demeaning strategy differs. Both implementations run in isolated processes and report peak RSS through `ru_maxrss`.

Peak physical memory use (three fixed effects, one covariate).

Design	λ_2 (share)	PyFixest MAP	PyFixest LSMR factor-pair
<i>100K observations</i>			
simple (well-connected)	0.622 (1.00)	354 MiB	411 MiB
difficult (near-nested)	8.35×10^{-6} (1.00)	357 MiB	383 MiB
<i>1M observations</i>			
simple (well-connected)	0.622 (1.00)	758 MiB	1,000 MiB
difficult (near-nested)	8.35×10^{-6} (1.00)	775 MiB	953 MiB

Note: For each regression, we record the largest amount of physical memory used by an isolated Python process, in MiB (2^{20} bytes). The table compares PyFixest OLS regressions using MAP or factor-pair LSMR. Each regression has one covariate and worker, firm, and year fixed effects. The results cover the simple and difficult designs at 100,000 and 1 million observations; they do not show how memory changes with graph structure. In the λ_2 (share) column, λ_2 is the second-smallest eigenvalue of the normalized Laplacian for the worker-firm graph. We compute it after removing fixed-effect levels observed only once; smaller values mean weaker connectivity. For a disconnected pair graph, we compute λ_2 in each connected component and report the smallest value. The number in parentheses is that component's share of retained observations.

The preconditioner adds 26-57 MiB at 100K observations and 178-242 MiB at 1M observations, but even at 1M the overhead remains modest relative to the full panel data footprint. The additional storage holds factor-pair co-occurrences, partition weights, and local approximate Cholesky factors.

References

- Abowd, John M., Francis Kramarz, and David N. Margolis. 1999. "High Wage Workers and High Wage Firms." *Econometrica* 67 (2): 251–333. <https://doi.org/10.1111/1468-0262.00020>.
- Arridge, Simon R., Marta M. Betcke, and Lauri Harhanen. 2014. "Iterated Preconditioned LSQR Method for Inverse Problems on Unstructured Grids." *Inverse Problems* 30 (7): 75009. <https://doi.org/10.1088/0266-5611/30/7/075009>.
- Berge, Laurent. 2018. *Efficient Estimation of Maximum Likelihood Models with Multiple Fixed-Effects: The R Package Fenmlm*. Technical Report Nos. 2018–13.
- Bergé, Laurent R., Kyle Butts, and Grant McDermott. 2026. "Fixest: A Fast and Feature-Rich Framework for Econometric Estimations in R." <https://doi.org/10.48550/arXiv.2601.21749>.
- Correia, Sergio. 2014. "Reghdfe: Stata Module to Perform Linear or Instrumental-Variable Regression Absorbing Any Number of High-Dimensional Fixed Effects." <https://github.com/sergiocorreia/reghdfe>.
- Correia, Sergio. 2017. *Linear Models with High-Dimensional Fixed Effects: An Efficient and Feasible Estimator*. Technical report. <https://hdl.handle.net/10.2139/ssrn.3129010>.
- Correia, Sergio, Paulo Guimarães, and Tom Zylkin. 2020. "Fast Poisson Estimation with High-Dimensional Fixed Effects." *The Stata Journal* 20 (1): 95–115. <https://doi.org/10.1177/1536867X20909691>.
- FixedEffectModels.jl Developers. 2026. "Fixedeffectmodels.jl: Fast Estimation of Linear Models with High Dimensional Categorical Variables." <https://github.com/FixedEffects/FixedEffectModels.jl>.
- Fong, David Chin-Lung, and Michael Saunders. 2011. "LSMR: An Iterative Algorithm for Sparse Least-Squares Problems." *SIAM Journal on Scientific Computing* 33 (5): 2950–71. <https://doi.org/10.1137/10079687X>.
- Frisch, Ragnar, and Frederick V. Waugh. 1933. "Partial Time Regressions as Compared with Individual Trends." *Econometrica* 1 (4): 387–401. <https://doi.org/10.2307/1907330>.
- Gao, Yuan, Rasmus Kyng, and Daniel A. Spielman. 2025. "AC(k): Robust Solution of Laplacian Equations by Randomized Approximate Cholesky Factorization." *SIAM Journal on Scientific Computing*. <https://rasmuskyng.com/papers/GKS25.pdf>.
- Gaure, Simen. 2013. "OLS with Multiple High Dimensional Category Variables." *Computational Statistics & Data Analysis* 66 : 8–18. <https://doi.org/10.1016/j.csda.2013.03.024>.
- Goldsmith-Pinkham, Paul. 2026. *Tracking the Credibility Revolution across Fields*. Technical report.
- Guimaraes, Paulo, and Pedro Portugal. 2010. "A Simple Feasible Procedure to Fit Models with High-Dimensional Fixed Effects." *The Stata Journal* 10 (4): 628–49. <https://doi.org/10.1177/1536867X1101000406>.
- Irons, Bruce M., and Richard C. Tuck. 1969. "A Version of the Aitken Accelerator for Computer Iteration." *International Journal for Numerical Methods in Engineering* 1 (3): 275–77. <https://doi.org/10.1002/nme.1620010306>.
- Jochmans, Koen, and Martin Weidner. 2019. "Fixed-Effect Regressions on Network Data." *Econometrica* 87 (5): 1543–60. <https://doi.org/10.3982/ECTA14605>.

- Lovell, Michael C. 1963. "Seasonal Adjustment of Economic Time Series and Multiple Regression Analysis." *Journal of the American Statistical Association* 58 (304): 993–1010. <https://doi.org/10.1080/01621459.1963.10480682>.
- PyFixest Developers. 2026. "Pyfixest: Fast High-Dimensional Fixed Effects Regression in Python." <https://github.com/py-econometrics/pyfixest>.
- Spielman, Daniel A., and Shang-Hua Teng. 2014. "Nearly Linear Time Algorithms for Preconditioning and Solving Symmetric, Diagonally Dominant Linear Systems." *SIAM Journal on Matrix Analysis and Applications* 35 (3): 835–85. <https://doi.org/10.1137/090771430>.
- Stammann, Amrei. 2018. "Fast and Feasible Estimation of Generalized Linear Models with High-Dimensional K-Way Fixed Effects." <https://doi.org/10.48550/arXiv.1707.01815>.
- Toselli, Andrea, and Olof Widlund. 2005. *Domain Decomposition Methods: Algorithms and Theory*. Springer. <https://doi.org/10.1007/b137868>.
- within Developers. 2026. "Within: High-Performance Fixed-Effects Solver for Econometric Panel Data." <https://github.com/py-econometrics/within>.
- Xu, Jinchao. 1992. "Iterative Methods by Space Decomposition and Subspace Correction." *SIAM Review* 34 (4): 581–613. <https://doi.org/10.1137/1034116>.
- Yang, Mei, Gul Karaduman, and Ren-Cang Li. 2024. "Flexible Modified LSMR for Least Squares Problems."